

LCD Display (SPI)

LCD Display (SPI)

1. Learning Objectives
 - SPI Protocol
 - SPI Hardware Interface
 - SPI Advantages
 - SPI Disadvantages
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. SPI Parameter Configuration
 3. Pin Parameter Configuration
 - LCD Screen
 - External Flash
 4. SPI Protocol Usage
 5. Write Program
 6. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn basic SPI communication knowledge.
2. Read data from flash and display it on the onboard LCD screen.

SPI Protocol

SPI protocol (Serial Peripheral Interface) is a commonly used synchronous serial communication protocol designed for high-speed, short-distance communication between microcontrollers and external devices. It supports full-duplex communication, meaning data can be sent and received simultaneously between master and slave devices. It is commonly used for memory chips, sensors, display drivers, wireless modules, and other devices.

SPI Hardware Interface

SPI protocol typically uses four lines for data transmission:

- **SCLK (Serial Clock):** Clock signal generated by the master device for synchronizing communication.
- **MOSI (Master Output Slave Input):** Master device sends data to slave device.
- **MISO (Master Input Slave Output):** Slave device sends data to master device.
- **SS/CS (Slave Select):** Master device activates the target slave device through chip select signal.

SPI Advantages

- Fast communication speed, suitable for high bandwidth applications.
- Supports full-duplex transmission, improving efficiency.
- Simple hardware implementation, direct interface.
- Supports multiple slave device connections.

SPI Disadvantages

- Lack of standardized error detection mechanism.
- Master device pin count increases with the number of slave devices.
- Communication distance is limited, usually used for onboard communication.

2. Hardware Setup

W25Q32 is a common serial flash device using SPI (Serial Peripheral Interface) protocol. It has high-speed read, write, and erase functions and is widely used in embedded systems, storage devices, routers, and other high-performance electronic devices. W25Q32 has a capacity of 32 Mbit (4 MB). The numbers in the model represent different capacity options, such as W25Q16, W25Q64, W25Q128, etc., to meet requirements in different application scenarios.

W25Q32 chip memory is allocated by sector and block. Specifically:

- **Sector:** Each sector is 4KB.
- **Block:** Each block contains 16 sectors, meaning one block is 64KB.

These storage unit structures make data management and storage operations more flexible, especially suitable for embedded systems and storage applications that require frequent read/write operations.

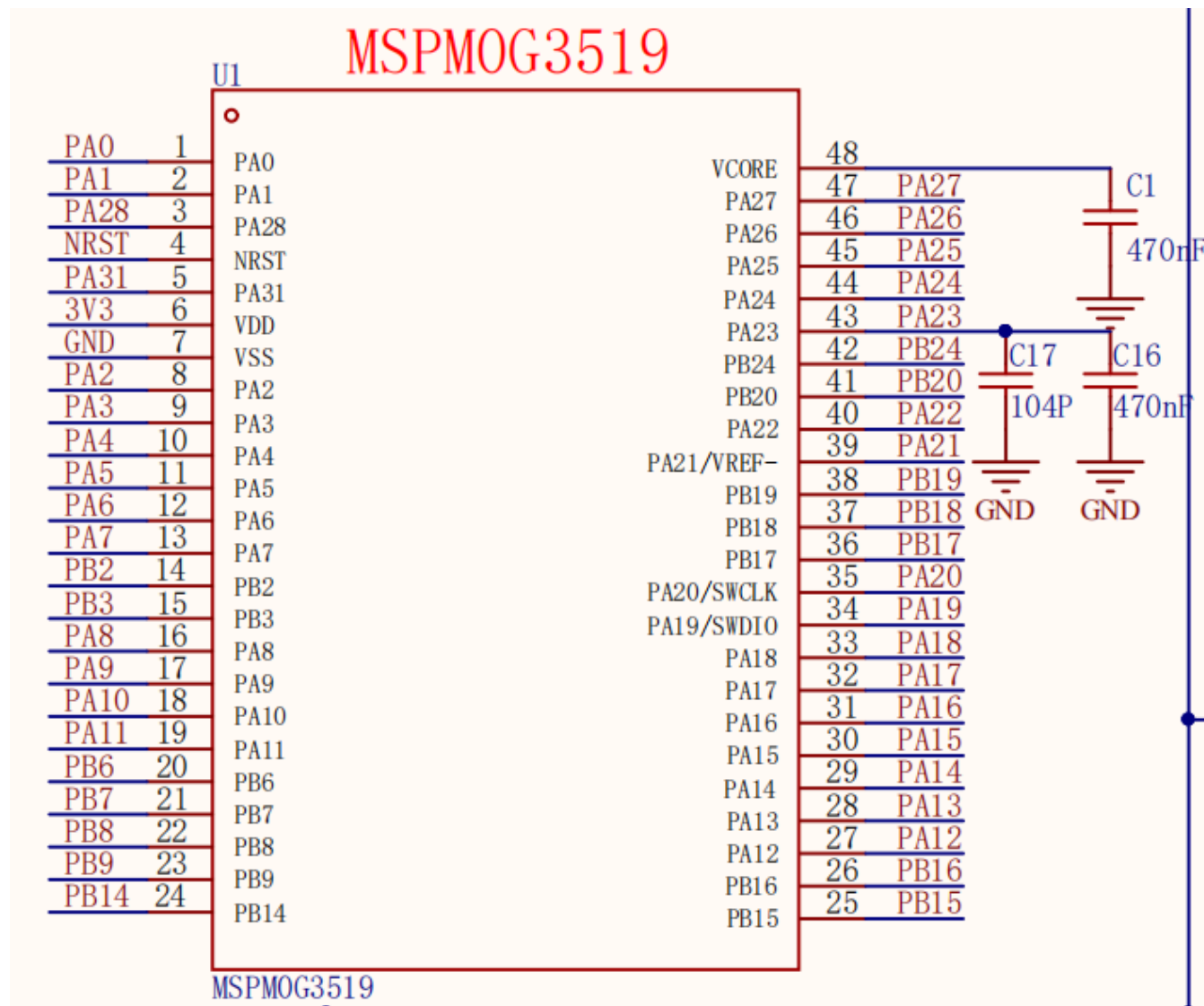
We use hardware SPI to drive W25Q32, so we need to determine if the pins we set have hardware SPI peripheral interface. According to the data manual, PB14~PB17 can be multiplexed as SPI1's 4 communication lines.

PZ PIN	PIN NAME/ IOMUX REG/ IOMUX ADDR	SIGNAL NAME	IOMUX PF	SIGNAL TYPE	BUFFER TYPE
48	PB14 PINCM31 0x40428078	PB14	1	IO	SDIO (standard)
		SPI1_CS3	2	IO	
		SPI1_POCI	3	IO	
		TIMG12_C1	4	IO	
		TIMA0_C0	5	IO	
		TIMG8_IDX	6	I	
		SPI0_CS3	7	IO	
49	PB15 PINCM32 0x4042807c	PB15	1	IO	SDIO (standard)
		UART7_TX	2	O	
		SPI1_PICO	3	IO	
		UART3_CTS	4	I	
		TIMG8_C0	5	IO	
		I2C2_SCL	7	IOD	
		TIMG7_C0	11	IO	
50	PB16 PINCM33 0x40428080	PB16	1	IO	SDIO (standard)
		UART7_RX	2	I	
		SPI1_SCK	3	IO	
		UART3_RTS	4	O	
		TIMG8_C1	5	IO	
		I2C2_SDA	7	IOD	
		TIMG7_C1	11	IO	
73	PB17 PINCM43 0x404280a8	PB17	1	IO	SDIO (standard)
		UART7_TX	2	O	
		SPI0_PICO	3	IO	
		I2C0_SCL	4	IOD	
		TIMA0_C2	5	IO	
		TIMG0_C0	6	IO	
		SPI1_CS1	7	IO	
		UART4_TX	8	O	
		TIMG14_C2	9	I	
		TIMA1_C0	11	IO	

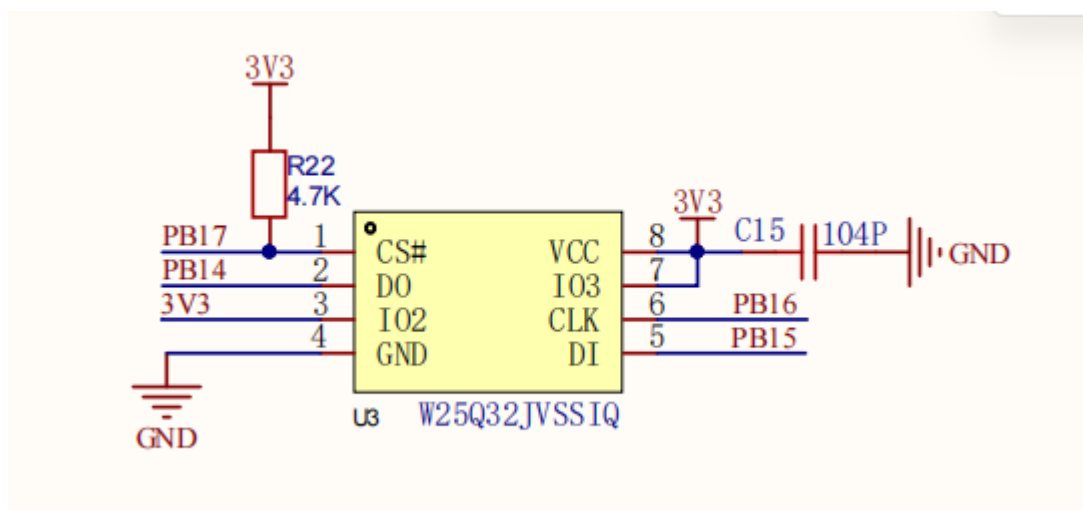
AI_4	(Non-IOMUX 1) 0	A
COMP1_IN2-	(Non-IOMUX 2) 0	A

This course does not require additional hardware. You can directly use the onboard LCD screen and external storage flash on the MSPM0G3519 motherboard.

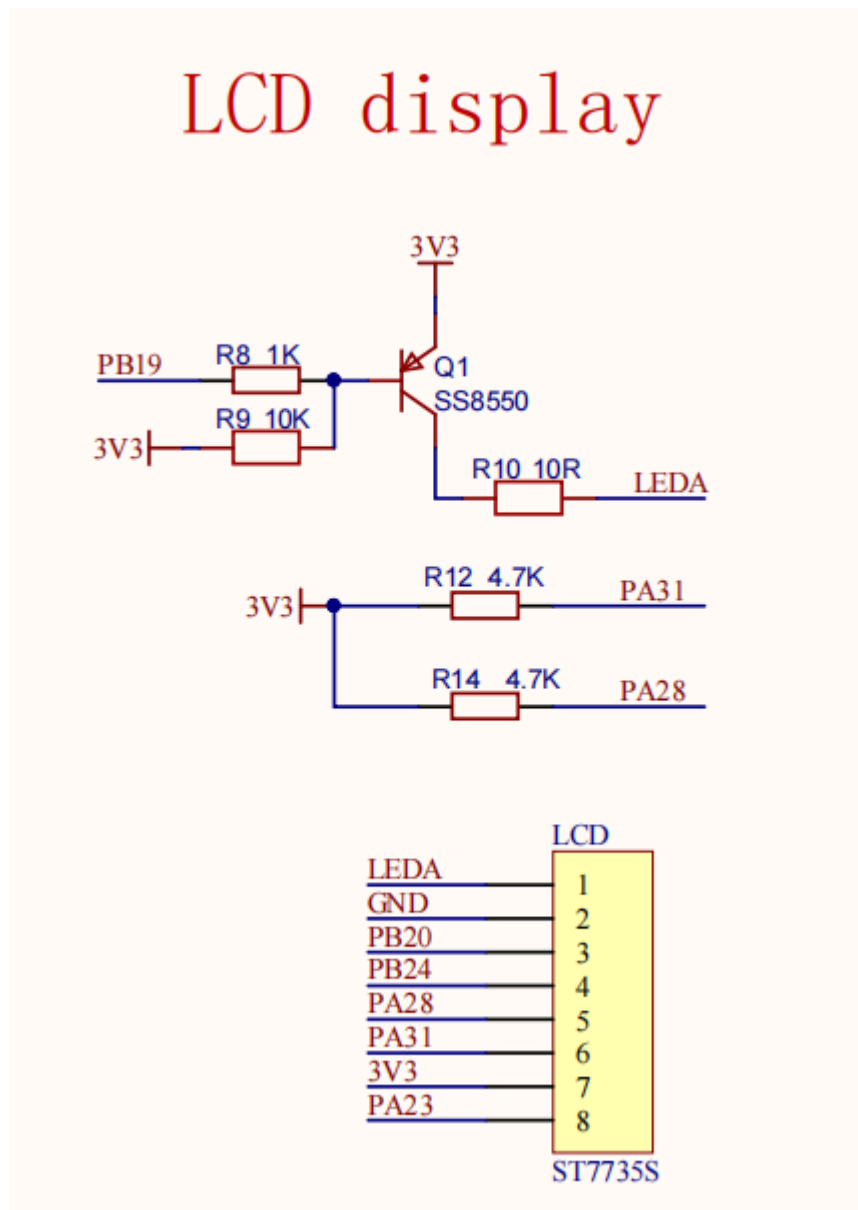
MSPM0G3519 Main Control Diagram:



W25Q32 Part Schematic:



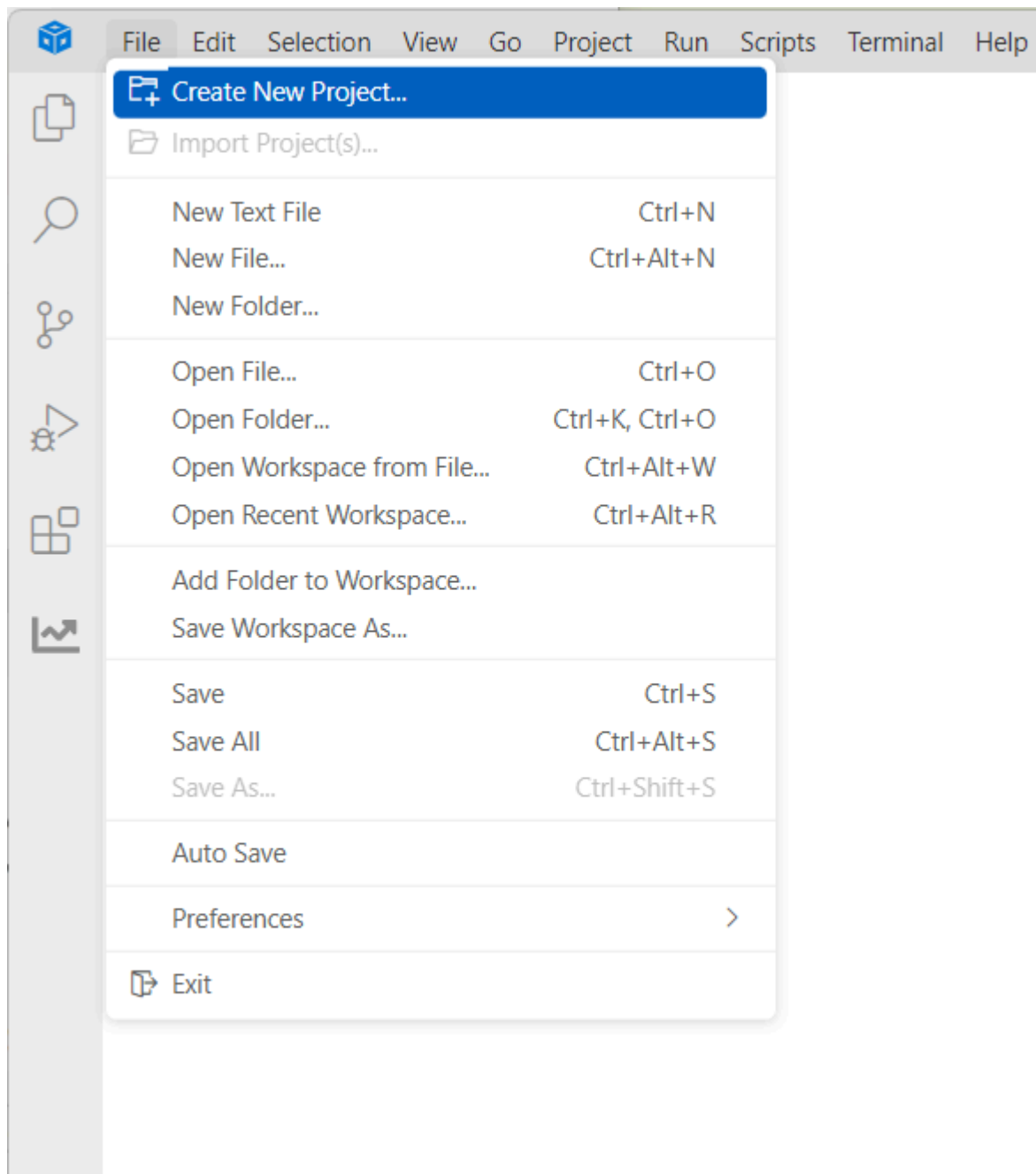
LCD Screen Part Schematic:

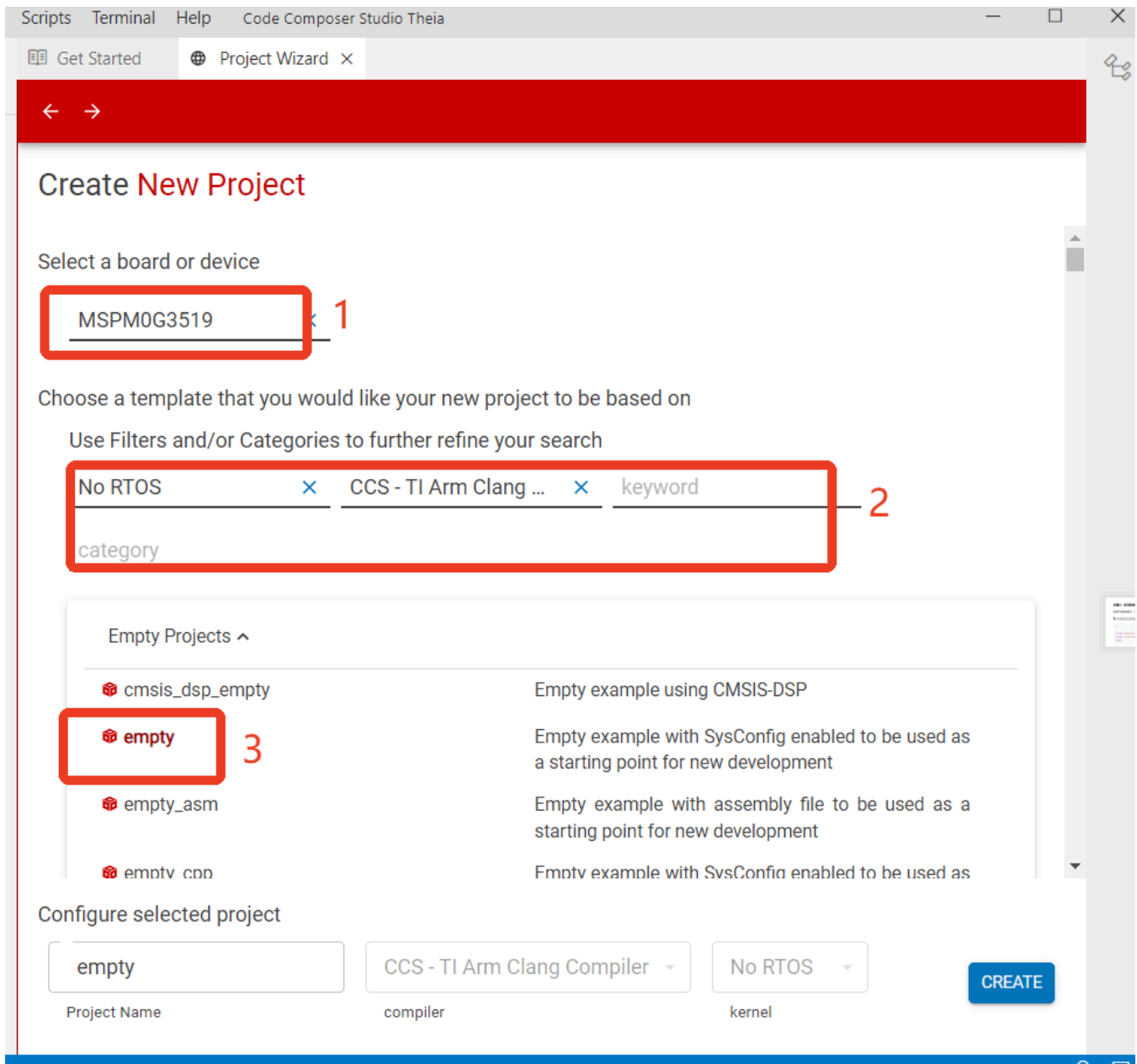


3. Experiment Steps

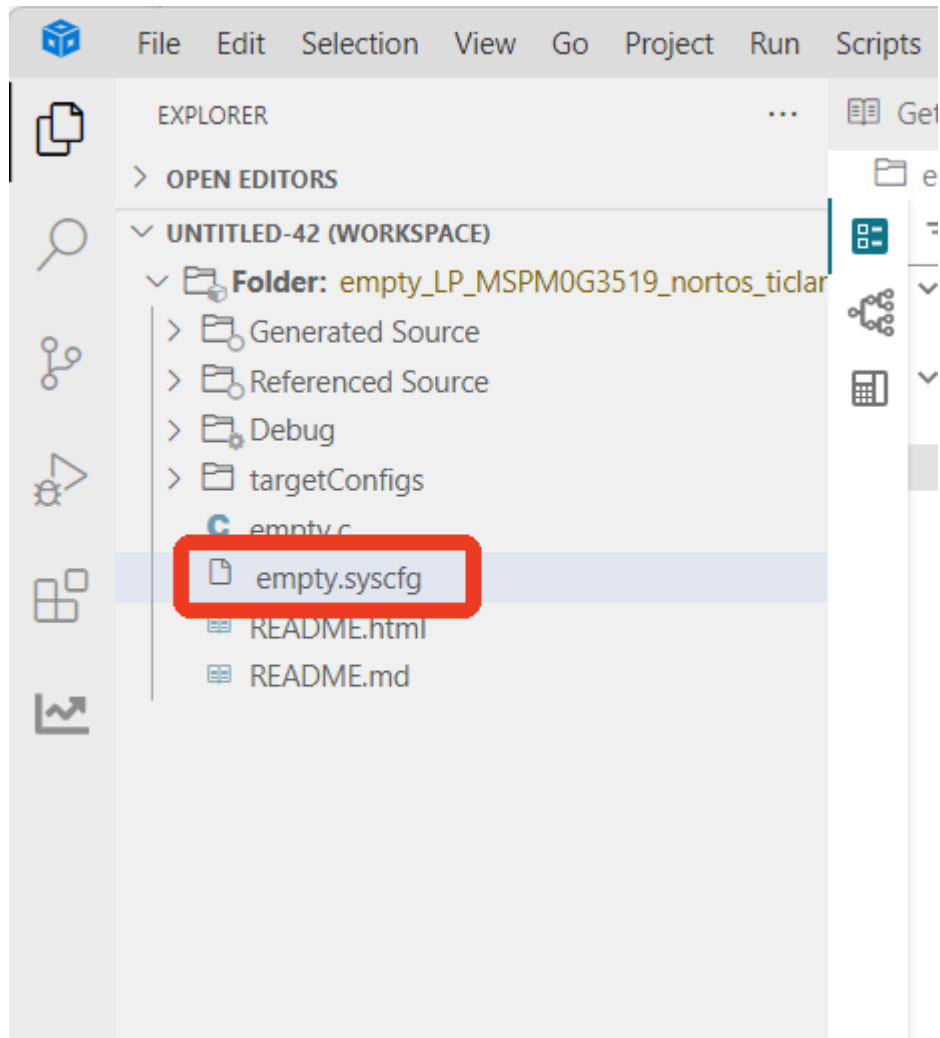
1. Open SYSCONFIG Configuration Tool

Create a new empty project empty in CCS.





Find and open the empty.syscfg file in the left workspace of CCS.



2. SPI Parameter Configuration

Find the SPI column on the left, click to enter, and add SPI peripheral.

Type Filter Text...

PROJECT CONFIGURATION (1)

Project Configur... 1/1 ✓ ⊕

MSPM0 DRIVER LIBRARY (7)

SYSTEM (9)

Board 1/1 ✓ ⊕

Configuration NVM ⊕

DMA ⊕

GPIO 1 ✓ ⊕

MATHACL ⊕

RTC ⊕

SYSCTL 1/1 ✓ ⊕

SYSTICK ⊕

WWDT ⊕

ANALOG (6)

ADC12 ⊕

COMP ⊕

DAC12 ⊕

GPAMP ⊕

OPA ⊕

VREF ⊕

COMMUNICATIONS (6)

I2C ⊕

I2C - SMBUS ⊕

MCAN ⊕

SPI ⊕

UART ⊕

UART - LIN ⊕

TIMERS (6)

Software > SPI

SPI (0 of 2 Added)

2

⊕ ADD

REMOVE ALL

Description

The **SPI module** provides an interface for synchronous serial communication with peripheral devices and other controllers.

Under *Basic Configuration* users can:

- Enable and configure SPI initialization parameters:
 - Mode select
 - Clock configuration
 - Frame configuration
 - Chip select

Under *Advanced Configuration* users can:

- Enable/configure parity mode
- Configure TX/RX FIFO thresholds
- Enable command data mode and/or internal loopback
- Configure sampling parameters

SPI Functional Block Diagram:

The diagram shows the SPI module's internal structure. It features a main 'SPI Control' block. Inside this block is a 'Clock Control' sub-block, which contains two smaller blocks: 'CLKSEL' and 'CLKDIV'. Three external clock inputs are shown on the left: 'SYSCLK', 'MFCLK', and 'LFCLK', each with an arrow pointing into the 'CLKSEL' block. An output line labeled 'SPI Clock' exits the 'CLKDIV' block to the right.

Click the **Add** button to add a SPI to your design

SPI (1 of 2 Added)

+

ADD

REMOVE ALL

✓

SPI

i

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

Name

SPI

Selected Peripheral

SPI1

Quick Profiles

SPI Profiles

Custom

Basic Configuration

SPI Initialization Configuration

Mode Select

Controller

Clock Configuration

Target Bit Rate (Hz)

16000000

Calculated Bit Rate

16000000.00

Calculated Error (%)

0

Frame Format

Motorola 3-wire

Clock Polarity

Low

Phase

Data captured on first clock edge

Frame Size (bits)

8

Bit Order

MSB

Advanced Configuration

Parity

Disabled

RX FIFO Threshold Level

RX FIFO contains >= 2 entries

TX FIFO Threshold Level

TX FIFO contains <= 2 entries

Communication Direction

PICO and POCI

Enable Packing

☐

In most SPI protocols, the chip select needs to remain low during the entire timing for sending and receiving. However, the SPI peripheral's chip select will go high after each frame is sent and received. Therefore, the CS chip select line needs to be controlled independently by the MCU's IO pin. The SPI peripheral's CS pin cannot be used. **Here, GPIO method (software method) is used to control the CS pin output.**

PinMux Peripheral and Pin Configuration		^	
SPI Peripheral	SPI1	▼	🔒
SPI SCLK (Clock)	PB16/4	▼	🔒
SPI PICO (Peripheral In, Controller Out)	PB15/3	▼	🔒
SPI POCI (Peripheral Out, Controller In)	PB14/2	▼	🔒

3. Pin Parameter Configuration

LCD Screen

Backlight **BLK**:

GPIO (6 Added) [?]

+

 ADD

REMOVE ALL

✓

 BLK

✓

 CS

✓

 DC

✓

 RES

✓

 MOSI

✓

 SCLK

📄

🗑️

📄

🗑️

📄

🗑️

📄

🗑️

📄

🗑️

📄

🗑️

Name

BLK

Port

PORTB

▼

Port Segment

Any

▼

Group Pins ^

1 added

+

 ADD

REMOVE ALL

✓

 PIN_19

🗑️

Name

PIN_19

Direction

Output

▼

Initial Value

Set

▼

IO Structure

Any

▼

Digital IOMUX Features

▼

Assigned Port

PORTB

▼

Assigned Port Segment

Any

▼

Assigned Pin

19

Interrupts/Events

▼

CS, DC, RES, MOSI, SCLK are as follows:

GPIO (6 Added)

ADD

REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

CS

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD

REMOVE ALL

PIN_23

Name

PIN_23

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

23

Interrupts/Events

GPIO (6 Added)

ADD

REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

DC

Port

PORTB

Port Segment

Any

Group Pins

1 added

ADD

REMOVE ALL

PIN_24

Name

PIN_24

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

24

Interrupts/Events

GPIO (6 Added)

ADD

REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

RES

Port

PORTB

Port Segment

Any

Group Pins

1 added

ADD

REMOVE ALL

PIN_20

Name

PIN_20

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

20

Interrupts/Events

GPIO (6 Added)

ADD

REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

SCLK

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD

REMOVE ALL

PIN_31

Name

PIN_31

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

31

Interrupts/Events

GPIO (6 Added)

ADD

REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

MOSI

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD

REMOVE ALL

PIN_28

Name

PIN_28

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

28

Interrupts/Events

External Flash

CS1:

✓ CS1

Name	CS1
Port	Any
Port Segment	Any

Group Pins

1 added

+

 ADD

REMOVE ALL

✓ PIN_17

Name	PIN_17
Direction	Output
Initial Value	Cleared
IO Structure	Any

Digital IOMUX Features

Assigned Port	PORTB
Assigned Port Segment	Any
Assigned Pin	17

Interrupts/Events

LaunchPad-Specific Pin	No Shortcut Used
------------------------	------------------

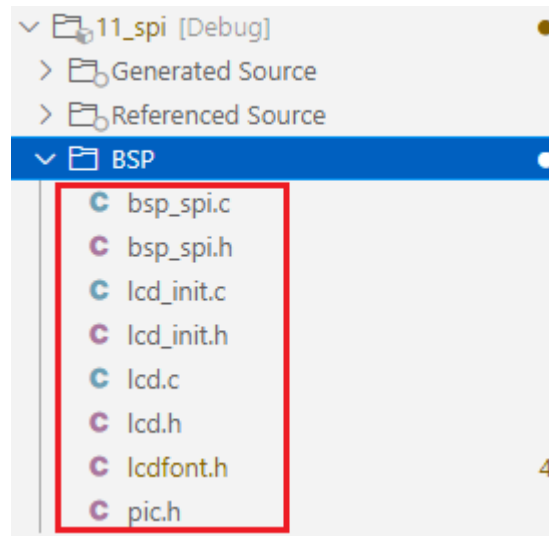
PinMux Peripheral and Pin Configuration

PIN_17	Any(PB17/14)
--------	--------------

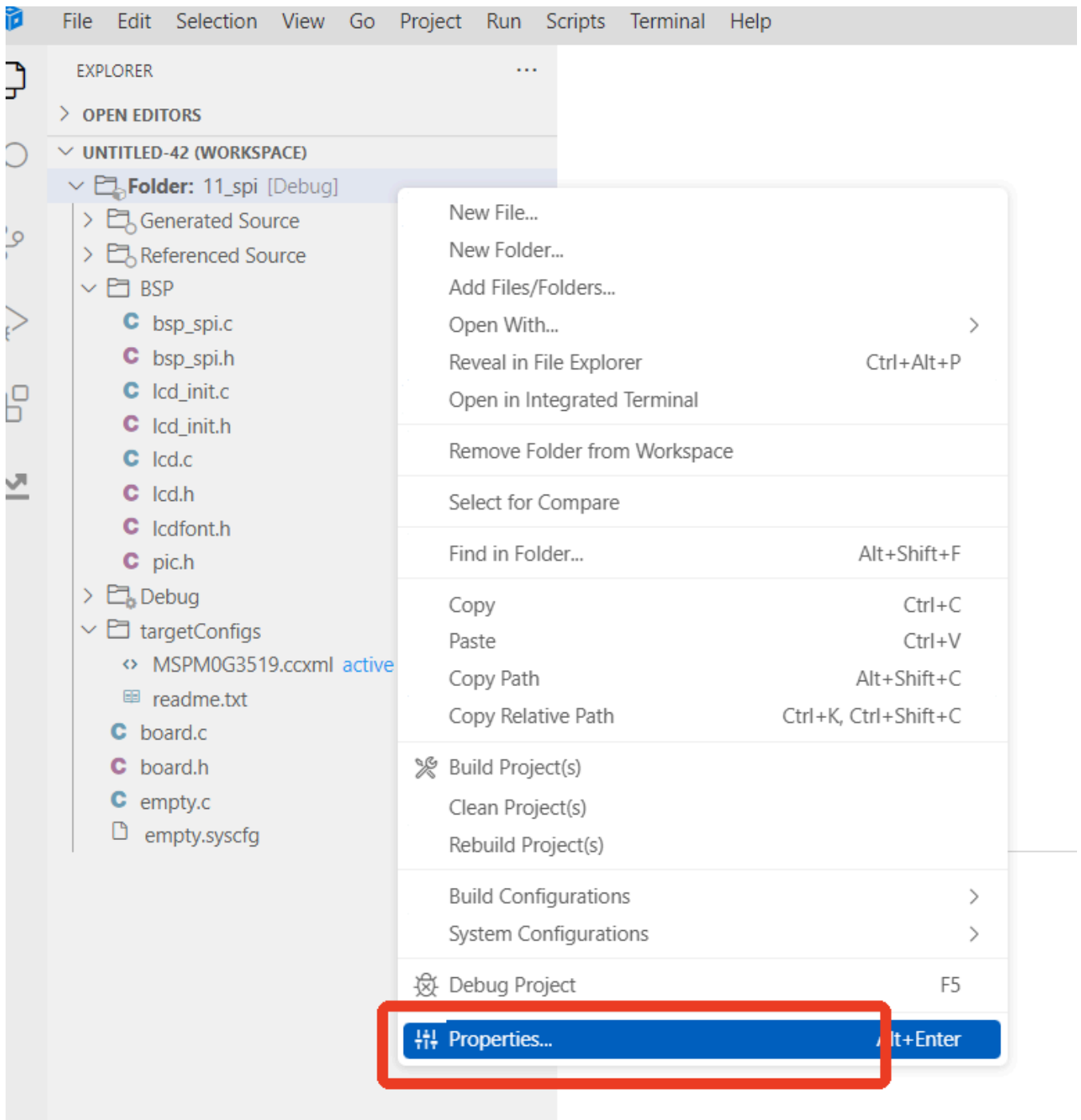
4. SPI Protocol Usage

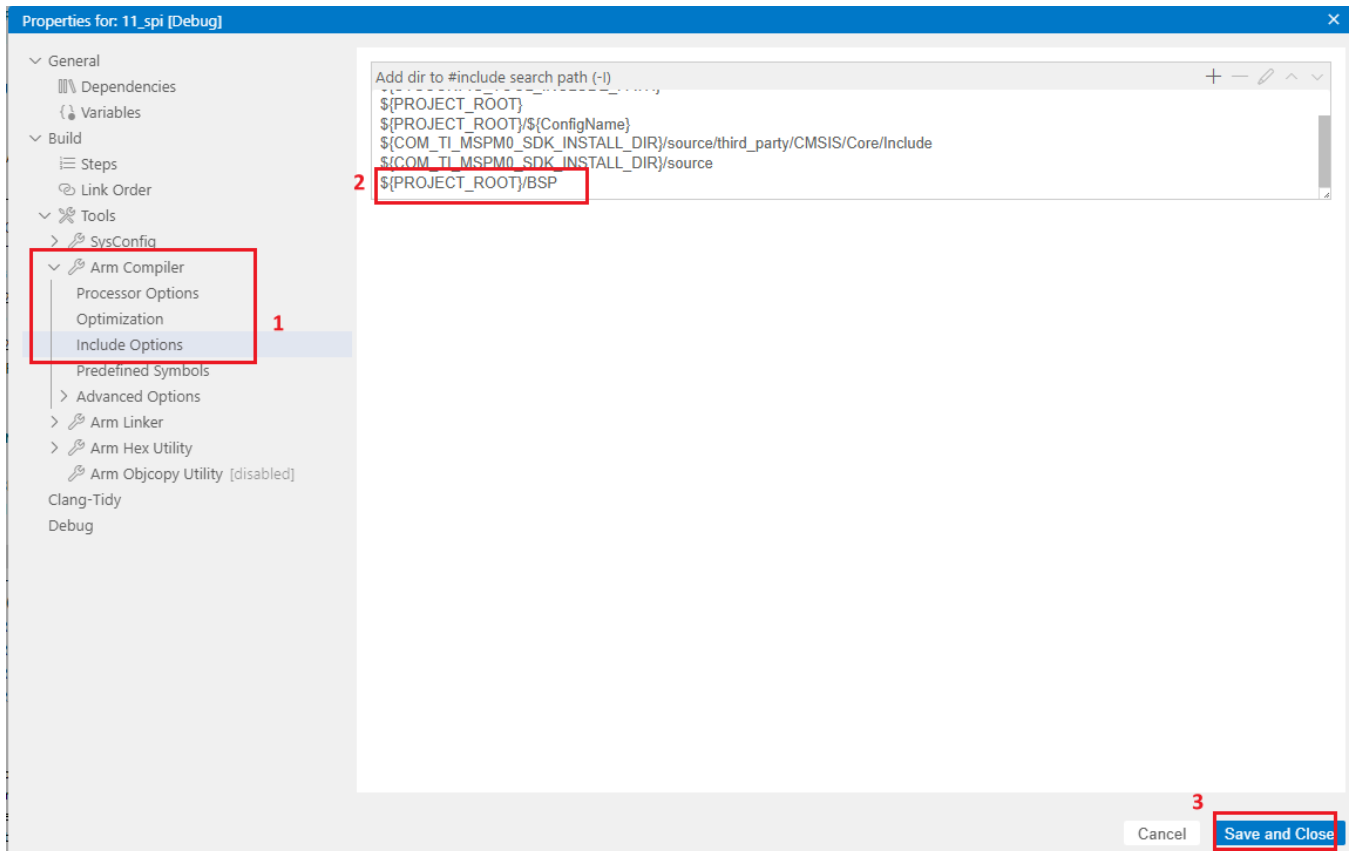
We create a new folder in the project folder: `BSP`. In the BSP folder, create four new files: `lcd.c`, `lcd.h`, `bsp_spi.c`, and `bsp_spi.h`.

The LCD display driver and font library are as follows, put in the BSP folder.



Update header file paths.





5. Write Program

Define LCD display operation interface and related functions.

lcd.h

```
#ifndef __LCD_H
#define __LCD_H
#include <stdint.h>

void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t color);//
Specify area fill color
void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color);// Draw a point at the specified
location
void LCD_DrawLine(uint16_t x1,uint16_t y1,uint16_t x2,uint16_t y2,uint16_t color);// Draw a
line at the specified position
void LCD_DrawRectangle(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,uint16_t color);//
Draw a rectangle at the specified location
void Draw_Circle(uint16_t x0,uint16_t y0,uint8_t r,uint16_t color);// Draw a circle at the
specified location

void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display Chinese character string
void LCD_ShowChinese12x12(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display a single 12x12 Chinese character
void LCD_ShowChinese16x16(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display a single 16x16 Chinese character
```

```

void LCD_ShowChinese24x24(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display a single 24x24 Chinese character
void LCD_ShowChinese32x32(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display a single 32x32 Chinese character

void LCD_ShowChar(uint16_t x,uint16_t y,uint8_t num,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display a character
void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);// Display String
uint32_t mypow(uint8_t m,uint8_t n);// Power
void LCD_ShowIntNum(uint16_t x,uint16_t y,uint16_t num,uint8_t len,uint16_t fc,uint16_t
bc,uint8_t sizey);// Display integer variables
void LCD_ShowFloatNum1(uint16_t x,uint16_t y,float num,uint8_t len,uint16_t fc,uint16_t
bc,uint8_t sizey);// Display variables with two decimal places

void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const uint8_t
pic[]);// Show image

// Brush Color
#define WHITE          0xFFFF
#define BLACK          0x0000
#define BLUE           0x001F
#define BRED           0XF81F
#define GRED           0xFFE0
#define GBLUE          0X07FF
#define RED            0xF800
#define MAGENTA        0xF81F
#define GREEN          0x07E0
#define CYAN           0x7FFF
#define YELLOW         0xFFE0
#define BROWN          0XBC40 // Brown
#define BRRED          0XFC07 // Brown red
#define GRAY           0X8430 // Gray
#define DARKBLUE       0X01CF  // Dark blue
#define LIGHTBLUE      0X7D7C  // Light blue
#define GRAYBLUE       0X5458  // Gray blue
#define LIGHTGREEN     0X841F  // Light green
#define LGRAY          0XC618  // Light gray (PANNEL), window background color
#define LGRAYBLUE      0XA651  // Light gray blue (middle layer color)
#define LBBLUE         0X2B12  // Light brown blue (inverted color of selected item)

#endif

```

Next is the LCD display driver.

lcd.c (only partial excerpt shown here, see project source code for details)

```

#include "lcd.h"
#include "lcd_init.h"
#include "lcdfont.h"
#include "board.h"

```

```

/*****
    Function description: Fill the specified area with color
    Input data: xsta, ysta starting coordinates
                xend, yend ending coordinates
                color the color to be filled
    Return value: None
*****/
void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t color)
{
    uint16_t i,j;
    LCD_Address_Set(xsta,ysta,xend-1,yend-1);// Set the display range
    for(i=ysta;i<yend;i++)
    {
        for(j=xsta;j<xend;j++)
        {
            LCD_WR_DATA(color);
        }
    }
}

/*****
    Function description: Draw a point at the specified position
    Input data: x, y coordinates of the point
                color color of the point
    Return value: None
*****/
void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color)
{
    LCD_Address_Set(x,y,x,y);// Set the cursor position
    LCD_WR_DATA(color);
}

...

```

bsp_spi.h

```

#ifndef _BSP_SPI_H__
#define _BSP_SPI_H__

#include "board.h"

// CS pin output control
// x=0 when output is low level
// x=1 when output is high level
#define SPI_CS(x)  ( (x) ? DL_GPIO_setPins(CS1_PORT,CS1_PIN_17_PIN) : DL_GPIO_clearPins(CS1_PORT,CS1_PIN_17_PIN) )

uint16_t w25Q32_readID(void);// Read the ID of w25Q32
void w25Q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte);      // w25Q32 write data
void w25Q32_read(uint8_t* buffer,uint32_t read_addr,uint16_t read_length);// w25Q32 Read Data

```

```
#endif
```

SPI initialization has been configured in SYSCONFIG, but we still need to prepare SPI read/write steps. To ensure successful sending and receiving of data, when sending, you need to ensure the data in the sending buffer is sent, i.e., the sending buffer is empty, before sending the next data; when receiving, you need to ensure there is data in the receiving buffer before receiving.

bsp_spi.c (only partial excerpt shown here, see project source code for details)

```
#include "bsp_spi.h"

uint8_t spi_read_write_byte(uint8_t dat)
{
    uint8_t data = 0;

    // Sending Data
    DL_SPI_transmitData8(SPI_INST, dat);
    // wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));
    // Receiving Data
    data = DL_SPI_receiveData8(SPI_INST);
    // wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));

    return data;
}

...
```

Then in the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"
#include "board.h"
#include "lcd_init.h"
#include "lcd.h"
#include "pic.h"
#include "bsp_spi.h"
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    unsigned char buff[10] = {0};

    // Development board initialization
    board_init();

    delay_ms(100); // waiting for deployment

    // Read the ID of W25Q32
    printf("ID = %X\r\n", W25Q32_readID());
}
```

```

// Read 5 bytes of data from address 0 to buff
w25q32_read(buff, 0, 7);

// Serial port outputs the read data
printf("buff = %s\r\n",buff);

// write 5 bytes of data ABCD to address 0
w25q32_write("Yahboom", 0, 7);
delay_ms(100);

// Read 5 bytes of data from address 0 to buff
w25q32_read(buff, 0, 7);

// Serial port outputs the read data
// printf("buff = %s\r\n",buff);

SYSCFG_DL_init();

LCD_Init();// LCD Initialization
LCD_Fill(0,0,LCD_W,LCD_H,WHITE);
LCD_ShowPicture(20,45,120,29,gImage_pic1);
LCD_ShowString(10,0,"Hello!",BLACK,WHITE,16,0);
LCD_ShowChinese(50,20,"亚博智能",BLACK,WHITE,16,0);

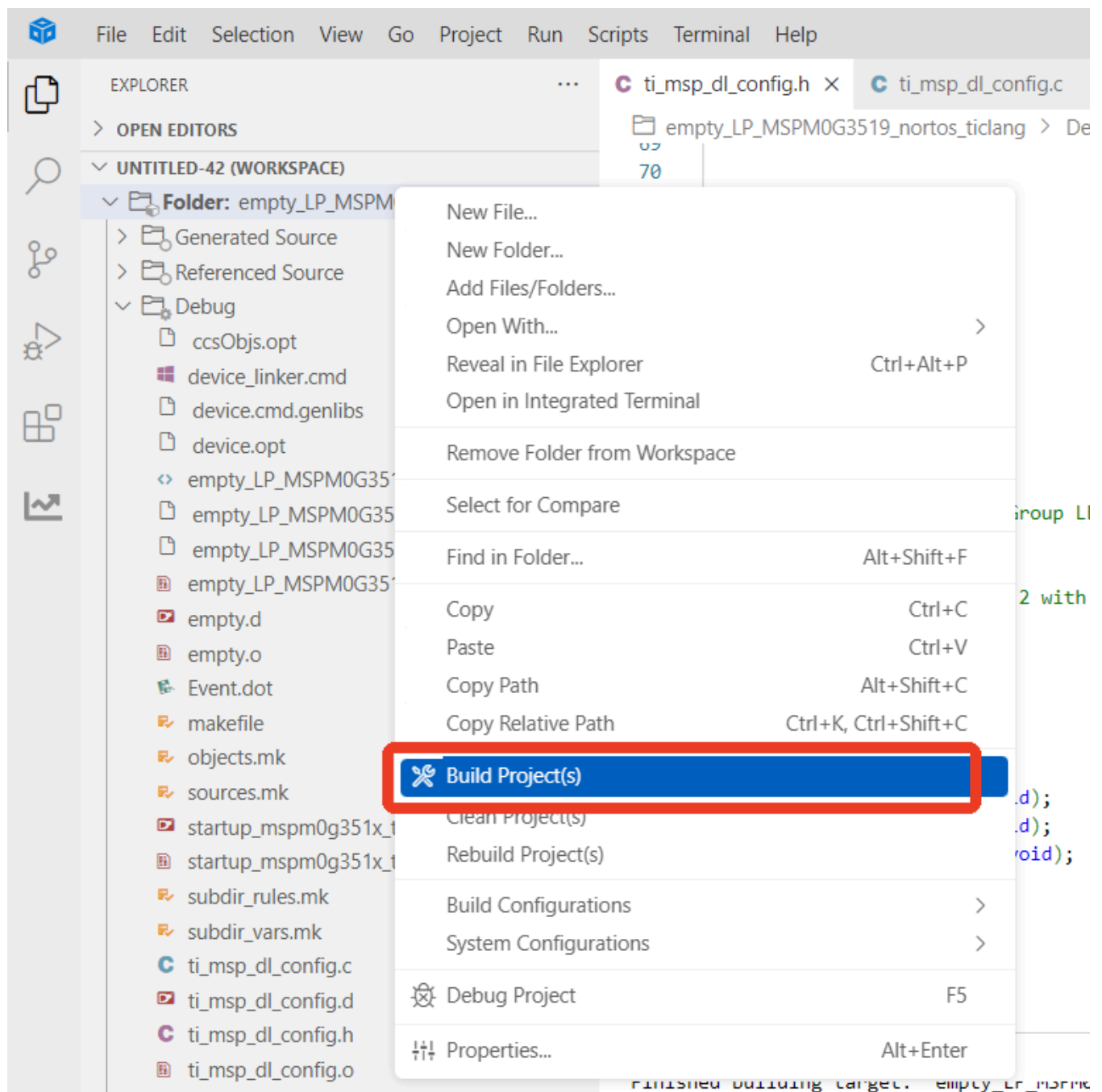
// Display the contents of buff on the same line
int x = 54; // Starting x coordinate
for (int i = 0; i < 7; i++)
{
    char str[2] = {buff[i], '\0'}; // Take one character at a time
    LCD_ShowString(x, 35, str,BLACK, WHITE, 16, 0); // Display Characters
    x += 8; // Each character is 8 wide and has an x-coordinate offset to the right.
}

while (1)
{

}
}

```

6. Compile



Compilation was successful. You can now download the program to the development board.

4. Program Analysis

- lcd.c

```

432 /*****
433 函数说明: 显示图片
434 入口数据: x,y起点坐标
435          length 图片长度
436          width 图片宽度
437          pic[] 图片数组
438 返回值: 无
439 Function description: Display image
440 Input data: x, y starting point coordinates
441          length Image length
442          width Image width
443          pic[] Image array
444 Return value: None
445 *****/
446 void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const uint8_t pic[])
447 {
448     uint16_t i,j;
449     uint32_t k=0;
450     LCD_Address_Set(x,y,x+length-1,y+width-1);
451     for(i=0;i<length;i++)
452     {
453         for(j=0;j<width;j++)
454         {
455             LCD_WR_DATA8(pic[k*2]);
456             LCD_WR_DATA8(pic[k*2+1]);
457             k++;
458         }
459     }
460 }

```

The `LCD_ShowPicture` function draws a picture at the specified starting point (x, y). It sets the display area according to the given length and width, and by iterating through the picture array, writes each pixel's color data to the LCD, completing the picture display.

```

309 /*****
310 函数说明: 显示字符串
311 入口数据: x,y显示坐标
312          *p 要显示的字符串
313          fc 字的颜色
314          bc 字的背景色
315          sizey 字号
316          mode: 0非叠加模式 1叠加模式
317 返回值: 无
318 Function description: Display string
319 Input data: x, y display coordinates
320          *p string to be displayed
321          fc color of the word
322          bc background color of the word
323          sizey font size
324          mode: 0 non-overlay mode 1 overlay mode
325 Return value: None
326 *****/
327 void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t bc,uint8_t sizey,uint8_t mode)
328 {
329     while(*p!='\0')
330     {
331         LCD_ShowChar(x,y,*p,fc,bc,sizey,mode);
332         x+=sizey/2;
333         p++;
334     }
335 }

```

The `LCD_ShowString` function displays a string at the specified coordinates (x, y). It calls the `LCD_ShowChar` function to draw each character in the string, adjusting character spacing according to font size. It supports overlay and non-overlay display modes.

```

152 /*****
153 函数说明: 显示汉字串
154 入口数据: x,y显示坐标
155           *s 要显示的汉字串
156           fc 字的颜色
157           bc 字的背景色
158           sizey 字号 可选 16 24 32
159           mode: 0非叠加模式 1叠加模式
160 返回值: 无
161 Function description: Display Chinese character string
162 Input data: x, y display coordinates
163           *s Chinese character string to be displayed
164           fc character color
165           bc character background color
166           sizey font size optional 16 24 32
167           mode: 0 non-overlay mode 1 overlay mode
168 Return value: None
169 *****/
170 void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t sizey,uint8_t mode)
171 {
172     while(*s!=0)
173     {
174         LCD_ShowChinese16x16(x,y,s,fc,bc,sizey,mode);
175         s+=2;
176         x+=sizey;
177     }
178 }

```

The `LCD_ShowChinese` function displays Chinese character strings at the specified coordinates `(x, y)`. It draws each Chinese character by calling the `LCD_ShowChinese16x16` function, adjusting the next Chinese character's display position according to font size. It supports overlay and non-overlay display modes.

- bsp_spi.c

```

172 /*****
173  * 函数名称: W25Q32_read
174  * 函数功能: 读取W25Q32的数据
175  * 传入参数: buffer=读出数据的保存地址  read_addr=读取地址  read_length=读去长度
176  * 函数返回: 无
177  * 作者: LC
178  * 备注: 无
179  * Function name: W25Q32_read
180  * Function function: Read W25Q32 data
181  * Input parameters: buffer = storage address of read data read_addr = read address read_length = read length
182  * Function return: None
183  * Author: LC
184  * Notes: None
185 *****/
186 void W25Q32_read(uint8_t* buffer,uint32_t read_addr,uint16_t read_length)
187 {
188     uint16_t i;
189     //拉低CS端为低电平 Pull the CS end to a low level
190     SPI_CS(0);
191     //发送指令03h Send instruction 03h
192     spi_read_write_byte(0x03);
193     //发送24位读取数据地址的高8位
194     // Send the high 8 bits of the 24-bit read data address
195     spi_read_write_byte((uint8_t)((read_addr)>>16));
196     //发送24位读取数据地址的中8位
197     // Send the middle 8 bits of the 24-bit read data address
198     spi_read_write_byte((uint8_t)((read_addr)>>8));
199     //发送24位读取数据地址的低8位
200     // Send the low 8 bits of the 24-bit read data address
201     spi_read_write_byte((uint8_t)read_addr);
202     //根据读取长度读取出地址保存到buffer中
203     // Read the address according to the read length and save it in the buffer
204     for(i=0;i<read_length;i++)
205     {
206         buffer[i]= spi_read_write_byte(0xFF);
207     }
208     //恢复CS端为高电平 Restore the CS end to a high level
209     SPI_CS(1);
210 }

```

The `w25q32_read` function reads data from the W25Q32 flash chip. The function first starts SPI communication by pulling the CS pin low, then sends the read command (0x03) and the high, middle, and low bytes of the read address. Then, according to the specified read length, it reads data byte by byte and stores it in the provided buffer (`buffer`). Finally, the function restores the CS pin to high level, ending the data transfer. This process ensures reading the specified length of data from the specified address.

```

123 /*****
124 * 函数名称: W25Q32_write
125 * 函数功能: 写数据到W25Q32进行保存
126 * 传入参数: buffer=写入的数据内容      addr=写入地址      numbyte=写入数据的长度
127 * 函数返回: 无
128 * 作者: LC
129 * 备注: 无
130 * Function name: W25Q32_write
131 * Function function: Write data to W25Q32 for storage
132 * Input parameters: buffer = data content to be written addr = write address numbyte = length of written data
133 * Function return: None
134 * Author: LC
135 * Notes: None
136 *****/
137 void W25Q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte)
138 {
139     unsigned int i = 0;
140     //擦除扇区数据 Erase sector data
141     W25Q32_erase_sector(addr/4096);
142     //写使能 Write enable
143     W25Q32_write_enable();
144     //忙检测 Busy detection
145     W25Q32_wait_busy();
146     //写入数据 Write data
147     //拉低CS端为低电平 Pull CS to low level
148     SPI_CS(0);
149     //发送指令02h Send instruction 02h
150     spi_read_write_byte(0x02);
151     //发送写入的24位地址中的高8位
152     // Send the high 8 bits of the 24-bit address to be written
153     spi_read_write_byte((uint8_t)((addr)>>16));
154     //发送写入的24位地址中的中8位
155     // Send the middle 8 bits of the 24-bit address to be written
156     spi_read_write_byte((uint8_t)((addr)>>8));
157     //发送写入的24位地址中的低8位
158     // Send the low 8 bits of the 24-bit address to be written
159     spi_read_write_byte((uint8_t)addr);
160     //根据写入的字节长度连续写入数据buffer
161     // Continuously write data buffer according to the length of the written byte
162     for(i=0;i<numbyte;i++)
163     {
164         spi_read_write_byte(buffer[i]);
165     }
166     //恢复CS端为高电平 Restore CS end to high level
167     SPI_CS(1);
168     //忙检测 Busy detection
169     W25Q32_wait_busy();
170 }

```

The `W25Q32_write` function writes data to the W25Q32 flash chip. First, the function erases the sector data at the target address. Then, it enables write operation and checks if the chip is busy. Next, the function sends the write command (0x02) and data address (24-bit address) to the chip through the SPI interface, and writes data byte by byte according to the specified write length. Finally, the function waits for the write operation to complete and restores the CS pin to high level, ensuring data is correctly written and the chip is in idle state.

- empty.c

```

11 int main(void)
12 {
13     unsigned char buff[10] = {0};
14
15     //开发板初始化 Development board initialization
16     board_init();
17
18     delay_ms(100); //等待部署 Waiting for deployment
19
20     //读取W25Q32的ID Read the ID of W25Q32
21     printf("ID = %X\r\n", W25Q32_readID());
22
23     //读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
24     W25Q32_read(buff, 0, 7);
25
26     //串口输出读取的数据 Serial port outputs the read data
27     printf("buff = %s\r\n", buff);
28
29     //往0地址写入5个字节长度的数据 ABCD Write 5 bytes of data ABCD to address 0
30     W25Q32_write("Yahboom", 0, 7);
31     delay_ms(100);
32
33     //读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
34     W25Q32_read(buff, 0, 7);
35
36     //串口输出读取的数据 Serial port outputs the read data
37     // printf("buff = %s\r\n", buff);
38
39     SYSCFG_DL_init();
40
41     LCD_Init(); //LCD初始化 LCD Initialization
42     LCD_Fill(0, 0, LCD_W, LCD_H, WHITE);
43     LCD_ShowPicture(20, 45, 120, 29, gImage_pic1);
44     LCD_ShowString(10, 0, "Hello!", BLACK, WHITE, 16, 0);
45     LCD_ShowChinese(50, 20, "亚博智能", BLACK, WHITE, 16, 0);
46
47     // 显示 buff 中的内容在同一行上 Display the contents of buff on the same line
48     int x = 54; // 起始 x 坐标 Starting x coordinate
49     for (int i = 0; i < 7; i++)
50     {
51         char str[2] = {buff[i], '\0'}; // 每次取一个字符 Take one character at a time
52         LCD_ShowString(x, 36, str, BLACK, WHITE, 16, 0); // 显示字符 Display Characters
53         x += 8; // 每个字符的宽度为 8, x 坐标向右偏移 Each character is 8 wide and has an x-coordinate offset to the right.
54     }
55
56     while (1)
57     {
58     }
59 }
60 }

```

The main function of this program is to initialize the development board, read and write data to the W25Q32 flash chip, and display the read content on LCD. First, the program initializes the development board and related hardware. Then, it reads the W25Q32 chip ID and outputs it through the serial port. Next, the program reads 7 bytes of data from the flash memory starting address (0 address) to the `buff` array and outputs this data through the serial port. Then, it writes the string "Yahboom" to flash address 0 and reads and displays the data again. Finally, the program initializes the LCD screen, displays welcome information and pictures, and displays the read `buff` data character by character on the same line on the screen.

5. Experimental Phenomena

After the program is downloaded, the LCD display screen will show "Hello!", "亚博智能", flash data, and the Yahboom logo on separate lines.

